Backend Architecture

Folder structure

```
c:\ocr\server\
├── .env                          # gitignored
├── .env.example
├── .gitignore
├── alembic.ini
├── pyproject.toml                # ruff / mypy / pytest config
├── requirements.txt
├── requirements-dev.txt
├── storage/                      # gitignored — local invoice blobs
│   └── invoices/{org_id}/{yyyy}/{mm}/{uuid}.pdf
├── alembic/
│   ├── env.py                    # async engine variant
│   ├── script.py.mako
│   └── versions/
├── tests/
│   ├── conftest.py               # async engine + transactional session fixtures
│   ├── factories.py
│   ├── unit/
│   │   ├── test_matching_engine.py
│   │   ├── test_normalization.py
│   │   └── test_scoring_bands.py
│   ├── integration/
│   │   ├── test_auth_flow.py
│   │   ├── test_invoice_upload.py
│   │   └── test_kb_learning.py
│   └── fixtures/
│       ├── invoices/*.pdf
│       └── odoo_responses/*.json
└── app/
    ├── __init__.py
    ├── main.py                   # app factory, lifespan, middleware, handlers
    │
    ├── api/
    │   ├── deps.py               # get_db, get_current_user, get_current_org,
    │   │                         # get_odoo_service, get_ocr_service, pagination
    │   └── v1/
    │       ├── router.py         # APIRouter aggregator
    │       └── endpoints/
    │           ├── auth.py
    │           ├── invoices.py   # upload / list / detail / file / confirm / push
    │           ├── vendors.py    # vendor KB CRUD
    │           ├── odoo.py       # PO + partner passthrough, connection test
    │           ├── organizations.py  # Odoo credential management
    │           └── stats.py      # dashboard aggregates
    │
    ├── core/
    │   ├── config.py             # pydantic-settings
    │   ├── security.py           # argon2 + PyJWT
    │   ├── crypto.py             # Fernet envelope for Odoo API keys
    │   ├── logging.py            # structlog config (JSON prod / console dev)
    │   ├── errors.py             # AppError hierarchy
    │   ├── middleware.py         # RequestContextMiddleware (request-id, timing)
    │   └── constants.py          # weights, thresholds, MIME allowlist
    │
    ├── db/
    │   ├── base.py               # DeclarativeBase + naming convention + type map
    │   ├── session.py            # engine, sessionmaker, get_db, session_scope
    │   └── base_class_imports.py # imports all models for Alembic autogenerate
    │
    ├── models/                   # see document 02
    ├── schemas/                  # see document 04
    ├── repositories/
    │   ├── base.py               # org-scoped generic CRUD
    │   ├── user_repository.py
    │   ├── organization_repository.py
    │   ├── vendor_kb_repository.py
    │   └── match_history_repository.py
```

```
│
├── services/                 # see document 03
│   ├── auth_service.py
│   ├── storage_service.py    # local FS now, S3 interface later
│   ├── ocr_service.py
│   ├── odoo_service.py
│   ├── matching_engine.py
│   ├── kb_service.py
│   └── invoice_service.py    # orchestrator
│
├── workers/                  # scaffolded, unused in MVP
│   ├── arq_worker.py
│   └── tasks.py
│
└── utils/
    ├── text.py               # normalize_company_name, normalize_description
    ├── money.py
    ├── dates.py
    └── files.py              # magic-byte sniffing, sha256, size guard
```

Why these layers

The dependency direction is strictly one way:

```
endpoints  →  services  →  repositories  →  models
     ↘           ↓
      schemas   utils/core
```

- `endpoints` parse and validate requests, call one service, and serialize. No business logic, no ORM queries.
- `services` own business rules. `matching_engine.py` is deliberately pure — it takes data structures and returns a score, touching neither the database nor the network — which is what makes it testable against fixtures and safe to tune.
- `repositories` own queries and, critically, **org scoping**. Enforcing `organization_id` in one base class rather than in fifty endpoint handlers is the difference between a tenancy bug being impossible and being inevitable.
- `schemas` are the wire contract, separate from `models` so a column rename does not silently become an API break.

`requirements.txt`

Versions verified against PyPI. Install with Python **3.12**.

```
# ---------------------------------------------------------------- web layer
fastapi==0.141.1
uvicorn[standard]==0.52.1          # uvloop auto-excluded on win32; httptools+watchfiles install fine
python-multipart==0.0.32           # required for UploadFile / multipart parsing

# ---------------------------------------------------------------- validation & settings
pydantic==2.13.4
pydantic-settings==2.15.0
email-validator==2.3.0             # enables pydantic EmailStr

# ---------------------------------------------------------------- database
SQLAlchemy[asyncio]==2.0.52
asyncpg==0.31.0
alembic==1.19.1
greenlet==3.5.5                    # hard requirement of the SQLAlchemy asyncio bridge

# ---------------------------------------------------------------- auth & crypto
PyJWT==2.13.0                      # NOT python-jose (unmaintained, CVE history)
argon2-cffi==25.1.0                # NOT passlib (unmaintained, breaks on new Python)
cryptography==50.0.0               # Fernet encryption of per-tenant Odoo API keys

# ---------------------------------------------------------------- AI / OCR
mistralai==2.9.2                   # v2: import from mistralai.client, NOT mistralai

# ---------------------------------------------------------------- matching
rapidfuzz==3.14.5

# ---------------------------------------------------------------- http & resilience
httpx==0.28.1
tenacity==9.1.4
anyio==4.14.2                      # to_thread.run_sync for blocking xmlrpc

# ---------------------------------------------------------------- observability
structlog==26.1.0
orjson==3.11.9                     # fast JSON for structlog + ORJSONResponse

# ---------------------------------------------------------------- files
aiofiles==25.1.0
filetype==1.2.0                    # magic-byte sniffing; python-magic needs a libmagic DLL
pypdfium2==5.12.1                  # page count / preview render; prebuilt Windows wheels
pillow==12.3.0                     # image normalization before OCR

# ---------------------------------------------------------------- optional async jobs (phase 2)
# arq==0.28.0
# redis==8.1.0
```

`requirements-dev.txt`:

```
-r requirements.txt

pytest==9.1.1
pytest-asyncio==1.4.0
pytest-cov==7.1.0
freezegun==1.5.5
ruff==0.16.2
mypy==2.3.0
types-aiofiles
```

> **Do not pin `starlette`.** It has gone 1.x and FastAPI 0.141.1 already constrains a compatible range. Pinning it produces a resolver conflict on the next FastAPI bump.
>
> **`uvicorn[standard]` is safe on Windows.** The `uvloop` extra carries a `sys_platform != 'win32'` marker, so pip skips it silently. Never add `uvloop` explicitly — it has no Windows wheel and will fail to build.

Settings — `app/core/config.py`

Every credential is a `SecretStr` so an accidental `print(settings)` or a logged traceback cannot leak it.

```python
from __future__ import annotations

import secrets
from functools import lru_cache
from pathlib import Path
from typing import Annotated, Any, Literal

from pydantic import Field, PostgresDsn, SecretStr, field_validator
from pydantic_settings import BaseSettings, NoDecode, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        # utf-8-sig, not utf-8: Windows editors (Notepad, VS Code "UTF-8 with BOM",
        # PowerShell Out-File) prepend a BOM that otherwise becomes part of the
        # first key name and silently breaks the first setting.
        env_file_encoding="utf-8-sig",
        case_sensitive=True,
        extra="ignore",
    )

    # ------------------------------------------------------------------ app
    PROJECT_NAME: str = "AP Invoice Automation API"
    API_V1_PREFIX: str = "/api/v1"
    ENVIRONMENT: Literal["local", "staging", "production"] = "local"
    DEBUG: bool = False
    LOG_LEVEL: str = "INFO"
    LOG_JSON: bool = False  # True in staging/production

    # ------------------------------------------------------------------ security
    SECRET_KEY: SecretStr = Field(
        default_factory=lambda: SecretStr(secrets.token_urlsafe(48))
    )
    JWT_ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
    REFRESH_TOKEN_EXPIRE_DAYS: int = 14
    # Generate with:
    #   python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
    CREDENTIAL_ENCRYPTION_KEY: SecretStr

    # NoDecode stops pydantic-settings from json.loads()-ing the raw env string
    # before our validator runs, which is what makes `A,B` (not '["A","B"]') work.
    BACKEND_CORS_ORIGINS: Annotated[list[str], NoDecode] = ["http://localhost:3000"]

    @field_validator("BACKEND_CORS_ORIGINS", mode="before")
    @classmethod
    def _parse_cors(cls, v: Any) -> list[str]:
        if isinstance(v, str):
            return [o.strip().rstrip("/") for o in v.split(",") if o.strip()]
        return v

    # ------------------------------------------------------------------ database
    POSTGRES_HOST: str = "localhost"
    POSTGRES_PORT: int = 5432
    POSTGRES_USER: str = "postgres"
    POSTGRES_PASSWORD: SecretStr = SecretStr("postgres")
    POSTGRES_DB: str = "ap_automation"
    DATABASE_URL: PostgresDsn | None = None  # explicit override wins

    DB_POOL_SIZE: int = 10
    DB_MAX_OVERFLOW: int = 20
    DB_POOL_RECYCLE_SECONDS: int = 1800
    DB_ECHO: bool = False

    @property
    def sqlalchemy_dsn(self) -> str:
        if self.DATABASE_URL:
            # Normalise: accept postgres://, postgresql://, postgresql+asyncpg://
            raw = str(self.DATABASE_URL)
```

```
            _, _, rest = raw.partition("://")
            return f"postgresql+asyncpg://{rest}"
        return (
            f"postgresql+asyncpg://{self.POSTGRES_USER}:"
            f"{self.POSTGRES_PASSWORD.get_secret_value()}@"
            f"{self.POSTGRES_HOST}:{self.POSTGRES_PORT}/{self.POSTGRES_DB}"
        )

    # ---------------------------------------------------------------- mistral
    MISTRAL_API_KEY: SecretStr
    MISTRAL_OCR_MODEL: str = "mistral-ocr-latest"
    MISTRAL_CHAT_MODEL: str = "mistral-large-latest"  # fallback extraction
    MISTRAL_TIMEOUT_MS: int = 180_000
    MISTRAL_MAX_RETRIES: int = 3
    # Document-level annotation is capped at 8 pages by the API.
    OCR_MAX_PAGES: int = 8

    # ---------------------------------------------------------------- odoo (dev fallback only)
    ODOO_URL: str | None = None            # https://mycompany.odoo.com
    ODOO_DB: str | None = None
    ODOO_USERNAME: str | None = None
    ODOO_API_KEY: SecretStr | None = None
    ODOO_TIMEOUT_SECONDS: int = 30
    ODOO_MAX_RETRIES: int = 3
    ODOO_PO_FETCH_LIMIT: int = 200         # candidate POs pulled per match run
    ODOO_PO_LOOKBACK_DAYS: int = 365

    # ---------------------------------------------------------------- uploads
    STORAGE_ROOT: Path = Path("storage")
    MAX_UPLOAD_BYTES: int = 20 * 1024 * 1024
    ALLOWED_UPLOAD_MIME: tuple[str, ...] = (
        "application/pdf", "image/png", "image/jpeg", "image/tiff", "image/webp",
    )

    # ---------------------------------------------------------------- matching
    AUTO_CONFIRM_THRESHOLD: float = 92.0   # UI shows green / one-click confirm
    REVIEW_THRESHOLD: float = 65.0         # below this, manual PO search
    MAX_CANDIDATES_RETURNED: int = 5


@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()  # type: ignore[call-arg]


settings = get_settings()
```

.env.example

```
ENVIRONMENT=local
DEBUG=true
LOG_JSON=false

SECRET_KEY=change-me-openssl-rand-hex-32
CREDENTIAL_ENCRYPTION_KEY=paste-a-fernet-key-here
BACKEND_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000

POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=ap_automation

MISTRAL_API_KEY=...

ODOO_URL=https://yourco.odoo.com
ODOO_DB=yourco
ODOO_USERNAME=bot@yourco.com
ODOO_API_KEY=...
```

Async database sessions — `app/db/session.py`

Two settings here are non-obvious and both prevent a class of production bug.

```python
from __future__ import annotations

from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

from sqlalchemy.ext.asyncio import (
    AsyncEngine, AsyncSession, async_sessionmaker, create_async_engine,
)

from app.core.config import settings

engine: AsyncEngine = create_async_engine(
    settings.sqlalchemy_dsn,
    echo=settings.DB_ECHO,
    pool_size=settings.DB_POOL_SIZE,
    max_overflow=settings.DB_MAX_OVERFLOW,
    pool_pre_ping=True,                      # survives Postgres idle-timeout kills
    pool_recycle=settings.DB_POOL_RECYCLE_SECONDS,
    connect_args={
        # asyncpg caches prepared statements per connection; PgBouncer in
        # transaction mode reuses server connections and blows up on them.
        # Harmless when connecting directly, mandatory behind PgBouncer.
        "statement_cache_size": 0,
        "server_settings": {"application_name": settings.PROJECT_NAME},
        "timeout": 15,
    },
)

SessionFactory = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,   # critical: lets you read model attributes after commit,
                              # inside the response serializer, without a re-SELECT on a
                              # closed session (which raises MissingGreenlet).
    autoflush=False,          # explicit flush; avoids surprise INSERTs mid-read
)


async def get_db() -> AsyncIterator[AsyncSession]:
    """FastAPI dependency.

    Contract: the *caller* commits. This dependency guarantees rollback on any
    escaping exception and always returns the connection to the pool.

    Why not auto-commit here: dependency teardown runs after the response has
    begun serializing, so a commit failure at that point cannot be turned into a
    clean 500 — the headers are already on the wire. Explicit commits inside the
    service layer keep the failure inside the handler, where it can be reported.
    """
    session = SessionFactory()
    try:
        yield session
    except Exception:
        await session.rollback()
        raise
    finally:
        await session.close()


@asynccontextmanager
async def session_scope() -> AsyncIterator[AsyncSession]:
    """For workers / CLI / lifespan, where there is no request to hang off.
    Here auto-commit IS correct — there is no response to corrupt."""
    session = SessionFactory()
    try:
        yield session
        await session.commit()
    except Exception:
        await session.rollback()
        raise
```

```
    finally:
        await session.close()
```

Error hierarchy — `app/core/errors.py`

One exception class per failure mode, each carrying its own HTTP status and a stable machine-readable `code` the frontend can branch on.

```
from __future__ import annotations

from typing import Any


class AppError(Exception):
    status_code: int = 500
    code: str = "internal_error"
    message: str = "An unexpected error occurred."

    def __init__(self, message: str | None = None, *, details: Any = None) -> None:
        self.message = message or self.message
        self.details = details
        super().__init__(self.message)


class NotFoundError(AppError):
    status_code, code, message = 404, "not_found", "Resource not found."


class PermissionDeniedError(AppError):
    status_code, code, message = 403, "forbidden", "Not permitted."


class AuthenticationError(AppError):
    status_code, code, message = 401, "unauthenticated", "Invalid credentials."


class ConflictError(AppError):
    status_code, code, message = 409, "conflict", "Resource conflict."


class ValidationError(AppError):
    status_code, code, message = 422, "validation_error", "Invalid input."


class UploadRejectedError(ValidationError):
    code, message = "upload_rejected", "File rejected."


# ---- external systems -----------------------------------------------------
class ExternalServiceError(AppError):
    status_code, code = 502, "external_service_error"


class OdooError(ExternalServiceError):
    code, message = "odoo_error", "Odoo request failed."


class OdooAuthError(OdooError):
    status_code, code, message = 502, "odoo_auth_failed", "Odoo authentication failed."


class OdooUnavailableError(OdooError):
    status_code, code, message = 503, "odoo_unavailable", "Odoo is unreachable."


class OdooNotConfiguredError(AppError):
    status_code, code, message = 400, "odoo_not_configured", "Connect Odoo first."


class OCRError(ExternalServiceError):
    code, message = "ocr_failed", "Invoice OCR failed."
```

```
class OCRExtractionError(OCRError):
    code, message = "ocr_extraction_failed", "Could not read invoice fields."
```

Structured logging — `app/core/logging.py`

```
from __future__ import annotations

import logging
import sys

import structlog

from app.core.config import settings


def configure_logging() -> None:
    shared = [
        structlog.contextvars.merge_contextvars,   # request_id/user_id auto-injected
        structlog.stdlib.add_log_level,
        structlog.stdlib.add_logger_name,
        structlog.processors.TimeStamper(fmt="iso", utc=True),
        structlog.processors.StackInfoRenderer(),
        structlog.processors.UnicodeDecoder(),
    ]
    renderer = (
        structlog.processors.JSONRenderer()
        if settings.LOG_JSON
        # Colors off on Windows: the default cp1252 console cannot render them
        # alongside accented vendor names without raising UnicodeEncodeError.
        else structlog.dev.ConsoleRenderer(colors=sys.platform != "win32")
    )
    structlog.configure(
        processors=[*shared, structlog.processors.format_exc_info, renderer],
        wrapper_class=structlog.make_filtering_bound_logger(
            logging.getLevelNamesMapping()[settings.LOG_LEVEL]
        ),
        logger_factory=structlog.PrintLoggerFactory(),
        cache_logger_on_first_use=True,
    )
    logging.basicConfig(level=settings.LOG_LEVEL, format="%(message)s", stream=sys.stdout)
    for noisy in ("uvicorn.access", "sqlalchemy.engine.Engine", "httpx", "httpcore"):
        logging.getLogger(noisy).setLevel(logging.WARNING)
```

Request context middleware — `app/core/middleware.py`

Binds a request id into `structlog.contextvars` so **every** log line emitted anywhere downstream carries it without being passed a logger.

```
from __future__ import annotations

import time
import uuid

import structlog
from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint
from starlette.requests import Request
from starlette.responses import Response

logger = structlog.get_logger(__name__)

REQUEST_ID_HEADER = "X-Request-ID"


class RequestContextMiddleware(BaseHTTPMiddleware):
    async def dispatch(
```

```
        self, request: Request, call_next: RequestResponseEndpoint
    ) -> Response:
        request_id = request.headers.get(REQUEST_ID_HEADER) or uuid.uuid4().hex
        structlog.contextvars.clear_contextvars()
        structlog.contextvars.bind_contextvars(
            request_id=request_id,
            method=request.method,
            path=request.url.path,
        )
        request.state.request_id = request_id

        started = time.perf_counter()
        try:
            response = await call_next(request)
        except Exception:
            logger.exception(
                "request.failed",
                duration_ms=round((time.perf_counter() - started) * 1000, 2),
            )
            raise
        duration_ms = round((time.perf_counter() - started) * 1000, 2)
        response.headers[REQUEST_ID_HEADER] = request_id
        response.headers["X-Process-Time-Ms"] = str(duration_ms)
        logger.info(
            "request.completed",
            status_code=response.status_code,
            duration_ms=duration_ms,
        )
        return response
```

Application factory — `app/main.py`

```
from __future__ import annotations

from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

import structlog
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import ORJSONResponse
from sqlalchemy import text
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.api.v1.router import api_router
from app.core.config import settings
from app.core.errors import AppError
from app.core.logging import configure_logging
from app.core.middleware import RequestContextMiddleware
from app.db.session import engine

logger = structlog.get_logger(__name__)


@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    configure_logging()
    settings.STORAGE_ROOT.mkdir(parents=True, exist_ok=True)

    # Fail fast on a bad DSN rather than 500-ing the first real request.
    async with engine.begin() as conn:
        await conn.execute(text("SELECT 1"))
    logger.info("startup.complete", env=settings.ENVIRONMENT)

    yield

    # MUST run: otherwise asyncpg connections leak on every --reload cycle.
    await engine.dispose()
    logger.info("shutdown.complete")


def create_app() -> FastAPI:
```

```
    app = FastAPI(
        title=settings.PROJECT_NAME,
        version="0.1.0",
        openapi_url=f"{settings.API_V1_PREFIX}/openapi.json" if settings.DEBUG else None,
        docs_url="/docs" if settings.DEBUG else None,
        redoc_url=None,
        default_response_class=ORJSONResponse,
        lifespan=lifespan,
    )

    # Order matters. Middleware is applied bottom-up, so adding CORS last means it
    # runs first — which is what keeps CORS headers on error responses. Without
    # this, a 500 reaches the browser as an opaque CORS failure and you debug the
    # wrong problem.
    app.add_middleware(RequestContextMiddleware)
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.BACKEND_CORS_ORIGINS,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
        expose_headers=["X-Request-ID", "Content-Disposition"],
    )

    # ------------------------------------------------------------- exception handlers
    @app.exception_handler(AppError)
    async def _app_error(request: Request, exc: AppError) -> ORJSONResponse:
        logger.warning("app.error", code=exc.code, detail=exc.message)
        return ORJSONResponse(
            status_code=exc.status_code,
            content={
                "error": {
                    "code": exc.code,
                    "message": exc.message,
                    "details": exc.details,
                },
                "request_id": getattr(request.state, "request_id", None),
            },
        )

    @app.exception_handler(RequestValidationError)
    async def _validation(
        request: Request, exc: RequestValidationError
    ) -> ORJSONResponse:
        return ORJSONResponse(
            status_code=422,
            content={
                "error": {
                    "code": "validation_error",
                    "message": "Request validation failed.",
                    "details": exc.errors(),
                },
                "request_id": getattr(request.state, "request_id", None),
            },
        )

    @app.exception_handler(StarletteHTTPException)
    async def _http(request: Request, exc: StarletteHTTPException) -> ORJSONResponse:
        return ORJSONResponse(
            status_code=exc.status_code,
            content={
                "error": {
                    "code": "http_error",
                    "message": str(exc.detail),
                    "details": None,
                },
                "request_id": getattr(request.state, "request_id", None),
            },
            headers=getattr(exc, "headers", None),
        )

    @app.exception_handler(Exception)
    async def _unhandled(request: Request, exc: Exception) -> ORJSONResponse:
        logger.exception("unhandled.exception")
        return ORJSONResponse(
            status_code=500,
            content={
```

```
                "error": {
                    "code": "internal_error",
                    # Never leak internals in production; show them in dev.
                    "message": f"{type(exc).__name__}: {exc}"
                    if settings.DEBUG
                    else "Internal server error.",
                    "details": None,
                },
                "request_id": getattr(request.state, "request_id", None),
            },
        )

    @app.get("/health", tags=["health"], include_in_schema=False)
    async def health() -> dict[str, str]:
        return {"status": "ok", "env": settings.ENVIRONMENT}

    @app.get("/health/ready", tags=["health"], include_in_schema=False)
    async def ready() -> dict[str, str]:
        async with engine.connect() as conn:
            await conn.execute(text("SELECT 1"))
        return {"status": "ready"}

    app.include_router(api_router, prefix=settings.API_V1_PREFIX)
    return app


app = create_app()
```

Every error response — validation, business, or crash — has the same shape, so the frontend needs exactly one parser:

```
{
  "error": { "code": "odoo_unavailable", "message": "...", "details": null },
  "request_id": "8f2c1e..."
}
```

Dependencies — `app/api/deps.py`

The pattern that makes org scoping automatic:

```
from __future__ import annotations

from typing import Annotated

from fastapi import Depends, Header
from fastapi.security import OAuth2PasswordBearer
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.config import settings
from app.core.errors import AuthenticationError, OdooNotConfiguredError
from app.core.crypto import decrypt_secret
from app.core.security import decode_token
from app.db.session import get_db
from app.models.organization import Organization
from app.models.user import User
from app.repositories.user_repository import UserRepository
from app.services.odoo_service import OdooCredentials, OdooService

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl=f"{settings.API_V1_PREFIX}/auth/login", auto_error=False
)

DbSession = Annotated[AsyncSession, Depends(get_db)]


async def get_current_user(
    db: DbSession,
    token: Annotated[str | None, Depends(oauth2_scheme)] = None,
    access_token_cookie: Annotated[str | None, Header(alias="Cookie")] = None,
```

```
) -> User:
    """Accepts the JWT from EITHER the Authorization header OR an access_token
    cookie. The cookie path is what lets the Next.js frontend keep the token
    httpOnly — see document 05, section 'Auth strategy'. Ten lines here removes
    an entire class of XSS risk in the browser."""
    raw = token or _cookie_value(access_token_cookie, "access_token")
    if not raw:
        raise AuthenticationError("Missing credentials.")

    payload = decode_token(raw, expected_type="access")
    user = await UserRepository(db).get_active(payload["sub"])
    if user is None:
        raise AuthenticationError("User no longer active.")
    return user


CurrentUser = Annotated[User, Depends(get_current_user)]


async def get_current_org(user: CurrentUser) -> Organization:
    return user.organization


CurrentOrg = Annotated[Organization, Depends(get_current_org)]


async def get_odoo_service(org: CurrentOrg) -> OdooService:
    """Builds a per-tenant Odoo client from the org's encrypted credentials."""
    if not org.odoo_configured:
        raise OdooNotConfiguredError()
    return OdooService(
        OdooCredentials(
            url=org.odoo_url,
            db=org.odoo_db,
            username=org.odoo_username,
            api_key=decrypt_secret(org.odoo_api_key_encrypted),
        )
    )
```

Because `get_odoo_service` derives its credentials from the authenticated user's organization, it is structurally impossible for one tenant's request to reach another tenant's Odoo.

Alembic

Leave `sqlalchemy.url` **empty** in `alembic.ini`; it is injected from settings so no secret lands in a tracked file.

```
# alembic/env.py
from __future__ import annotations

import asyncio
from logging.config import fileConfig

from alembic import context
from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import async_engine_from_config

from app.core.config import settings
from app.db.base import Base
import app.db.base_class_imports  # noqa: F401  — registers every model on Base.metadata

config = context.config
config.set_main_option("sqlalchemy.url", settings.sqlalchemy_dsn)

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata


def do_run_migrations(connection: Connection) -> None:
```

```
    context.configure(
        connection=connection,
        target_metadata=target_metadata,
        compare_type=True,              # detect VARCHAR(50) -> VARCHAR(120)
        compare_server_default=True,
        render_as_batch=False,          # Postgres: batch mode not needed
    )
    with context.begin_transaction():
        context.run_migrations()


def run_migrations_offline() -> None:
    context.configure(
        url=settings.sqlalchemy_dsn,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )
    with context.begin_transaction():
        context.run_migrations()


async def run_async_migrations() -> None:
    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,        # one-shot CLI: never hold a pool
    )
    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)
    await connectable.dispose()


if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_async_migrations())
```

Two things Alembic will **not** do for you:

- **Enum values.** `ALTER TYPE ... ADD VALUE` is never autogenerated. Adding an `InvoiceStatus` member requires a hand-written `op.execute("ALTER TYPE invoice_status ADD VALUE 'x'")`.
- **Functional indexes.** A `lower(email)` unique index must be written by hand as `op.create_index("uq_users_email_lower", "users", [sa.text("lower(email)")], unique=True)`.

Add `op.execute("CREATE EXTENSION IF NOT EXISTS pgcrypto")` at the top of the first revision's `upgrade()`. It is a no-op on PostgreSQL 13+, where `gen_random_uuid()` is built in, and harmless insurance otherwise.

Running it

```
cd C:\ocr\server
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Always pass the app as the import string `"app.main:app"`, never as an object. On Windows `--reload` spawns a fresh process rather than forking, so the module is re-imported from scratch in the child — which is also why nothing expensive may run at import time, and why the startup DB ping lives in `lifespan` rather than at module level.